

Explicação do projeto Projeto-React

Este projeto é uma API REST em Node.js, escrita em TypeScript, com Express e TypeORM para gerenciar cadastros relacionados a usuários e produtos em um banco MySQL.

Apesar do nome 'Projeto-React', o código não contém frontend em React. O que existe aqui é um backend, ou seja, uma API que expõe endpoints para criar, listar, buscar, atualizar e excluir registros.

A aplicação oferece operações CRUD para cinco recursos principais: situações de usuário, usuários, categorias de produto, situações de produto e produtos.

Esses dados são persistidos em tabelas relacionais no MySQL e a API responde sob o prefixo /api.

O que o projeto faz

A API serve como backend para operações de cadastro e consulta de: usuários, situações de usuário, categorias de produto, situações de produto e produtos. Em outras palavras, ela funciona como um sistema de gestão de dados para um pequeno catálogo de usuários e itens de estoque/produtos.

Como o projeto funciona

1. Inicialização da aplicação

O ponto de entrada está em `src/index.ts`. Ele importa `reflect-metadata`, carrega as variáveis do ambiente, cria a instância do Express, monta as rotas em /api e chama `AppDataSource.initialize()` e `AppDataSource.runMigrations()` antes de abrir o servidor.

2. Configuração do banco de dados

O arquivo `src/data-source.ts` centraliza a configuração do TypeORM. Ele cria um `DataSource` para MySQL com host, porta, usuário, senha, nome do banco e entidades a serem mapeadas. O projeto exige `DB_DIALECT=mysql`, então o ambiente foi pensado para MySQL específico.

3. Entidades e relacionamentos

As entidades estão em `src/entities`. Existe uma tabela `Situation` para status de usuário, `User` para os usuários, `ProductCategory` para categorias, `ProductSituation` para estados do produto e `Product` para o cadastro dos produtos. O relacionamento principal mostra que cada usuário pertence a uma situação e cada produto pertence a categoria e situação do produto.

4. Migrations

O schema do banco é criado em `src/migrations/InitialSchema.ts`. Esse arquivo cria as tabelas `situations`, `product_categories`, `product_situations`, `users` e `products` e define chaves estrangeiras para manter integridade entre registros.

5. Rotas REST

- GET /api/situations
- GET /api/users/:id
- POST /api/users
- PATCH /api/products/:id
- DELETE /api/product-categories/:id

6. Controlador e serviço genéricos

Os arquivos `src/controllers/crudController.ts` e `src/services/crudService.ts` encapsulam a lógica repetitiva do CRUD. O controlador valida corpo, paginação, ids e campos obrigatórios; o serviço persiste dados e resolve relacionamentos do TypeORM. Isso evita duplicação de código para cada entidade.

7. Seeds iniciais

O script `src/seed.ts` insere dados básicos de exemplo, como Ativo/Inativo, Geral/Eletrônicos/Alimentos e Disponível/Indisponível. Isso deixa o banco pronto para testes e demonstração.

Fluxo de execução

1. O servidor inicia via `src/index.ts`.
2. O TypeORM conecta ao MySQL.
3. As migrations criam as tabelas.
4. As rotas recebem as requisições.
5. O controlador valida as entradas e chama o serviço.
6. O serviço grava ou recupera dados do banco.
7. A API responde em JSON.

Exemplo de criação de usuário

```
POST /api/users { "name": "Maria", "email": "maria@email.com", "situationId": 1 }
```

Exemplo de criação de produto

```
POST /api/products { "name": "Teclado", "productSituationId": 1, "productCategoryId": 2 }
```

Conclusão

Em resumo, o projeto é um backend de CRUD para gerenciamento de usuários e produtos em MySQL. Ele usa padrões de arquitetura REST, TypeORM, Express e migrations para organizar, validar, persistir e consultar os dados. A principal funcionalidade é oferecer endpoints padronizados para manutenção de cadastros, sem depender de um frontend React.